

Shell Programming Secrets

Nobody Talks About



Most tutorials about shell programming are often part of larger guides on Linux. They gloss over the numerous ways that your code might work but still fail under certain circumstances. Given that shell scripts are used to manage billions of dollars of assets, it is important to learn how to write clean and safe code with them.

Last year, I wrote a book on Linux command-line tips and tricks, and made several updates to it. Annoyingly, I continue to discover something new and important about the Bash shell program almost every week. I did not want this happening after I had ordered my author copy. The discoveries made me wonder what I have been doing all these years without knowing these bash secrets.

***sh* and *bash* are not the same**

The Bourne shell (*sh*) program began its life in the 70s with the UNIX operating system. Ubuntu Linux continues to have this old shell alongside its new avatar, the Bourne-Again shell (*bash*). Try *bash -version* in the command line, it

will display its version number. Try *sh -version*, you get an error. The two are different. While *sh* remains an ancient relic, *bash* continues to be developed and has a lot more features.

It was my practice (in the late 90s) to run my shell scripts in SCO UNIX with the *sh* command. I continued this in Ubuntu and found that a lot of online script examples did not work with it. (As a security measure, I never give the extension *.sh* or the *+x* permission to my scripts. My scripts remain anonymous with an innocuous *.txt* extension.)

Aware of this problem, a lot of script authors place a comment *#!/bin/bash* on the first line. This comment ensures that the script will be run with *bash* even if it is invoked with *sh*.

Some overzealous fanatics use the comment *#!/usr/bin/env bash* instead as a more failsafe measure. They say that *bash* may not always be at */bin*, so it is better to make *env* to find it. By this, they assume that *env* will always be found at */usr/bin*. Seems overkill to me. If you are on Ubuntu, as most people are, then *#!/bin/bash* should do fine.

***if* statements are not what they seem to be**

The shell's *if* statement is very unusual.

```
if test-expression; then
    statements;
```